4 Functions

Functions First

Create a program called **functions_first**. The program should have a function called **print_powers** that prints the first 20 powers of two to the console. Call the **print_powers** function from the *main function*.

Please **use a for loop** in your program.

Example output:

```
1 2 4 8 16 32 64 128 256 512 1024 2048 4096 8192 16384 32768 65536 131072 262144 524288
```

Functions Second

Create a program called **functions_second**. The program should have a function called **print_rectangle** that takes the height and width of a rectangle as arguments. The function should print a rectangle using the character "*" as shown in the example output.

The *main function* should first input the height and width of a rectangle and then then call the **print_rectangle** function with the inputted values.

Example output:

```
Enter height: 4
Enter width: 10
**********
**********
**********
**********
```

Functions Third

Create a program called **functions_third.** The program should have a function called **compute_discount** that takes *price* in euros and *discount percentage* as arguments. The function should compute and return the amount of discount in euros.

The *main function* should first input price in euros and discount percentage from the user and then call the **compute_discount** function with the inputted values. Finally, the *main function* should print the discount in euros.

NB! The **compute_discount** function should not print anything.

Example output:

```
Enter price: 1500.00
Enter discount percentage: 22.5
The discount is 337.50 euros
```

Functions Earnings

Create a program called **functions_earnings**. The program should have a function called **compute_earnings** that takes *hourly wage* and *hours worked* as arguments. The function should compute and return the earnings.

The ***overtime pay*** is time-and-a-half after 40 work hours.

The *main function* should first input *hourly wage* and *hours* (integer) from the user and then call the **compute_earnings** function with the inputted values. Finally, the *main function* should print the earnings with two decimal places.

NB! The **compute_earnings** function should not print anything.

Example output:

```
Enter wage: 24.25
Enter hours: 50
The earnings are 1333.75
```

Functions Add

Create a program called **functions_add**. The program should have a function called **add** that takes two floats as arguments. The function should return the sum of the floats as an integer. When rounding is needed, then half should be rounded up. (*One trick is to add 0.5 before type conversion*)

The *main function* should first input two floats from the user and then call the **add** function with the inputted values. Finally, the *main function* should print the integer, which the **add** function returns.

NB! The **add** function should not print anything.

Example output:

```
Enter a float: 2.2
Enter a float: 3.3
6
```

Functions Add 2 (Please see 'Conditional execution of the main function' in the course material)

Create a program called **functions_add_2**. Copy/paste all the code from your **functions_add** program to the new program. Then modify the new program as follows:

```
if __name__ == "__main__":
    main()
```

In addition to running the **functions_add_2 program** normally, we can now do some *unit testing* where the test module calls your **add** function without running your program. Please copy/paste your code to Viope. This time, when you click on the Run button, Viope runs the test module and Viope shows the *unit test report* (instead of your program's normal output) in the test execution console. Therefore, we do not have the example output in this exercise description.

Functions Pyramid (*This is a grade level 3-5 exercise*)

Create a program called **functions_pyramid**. The program should have a function called **print_pyramid** that takes the height of the pyramid as argument. The function should print a pyramid using the character "*" as shown in the example output.

The *main function* should first input the height of the pyramid and then call the **print_pyramid** function with the inputted values.

Example output:

```
Enter pyramid height: 4
   *
  ***
 *****
*******
```

Hint: `print(5 * "X")` prints XXXXX

If we take advantage of this operation, the whole loop that prints the pyramid can be written on two lines of code.

Functions Double Factorials (*This is a grade level 4-5 exercise*)

Create a program called **functions_double_factorials**. The program should have a function called **double_factorial** that takes an integer as argument and returns the double *factorial* of the integer. See the previous "Double Factorial" exercise for more details. The **double_factorial** function should not print anything.

The *main function* should print double factorials of 0 – 9 as shown in the example output. To determine the double factorial of each number, the main function should call the **double_factorial** function. Please use the for loop in the main function.

Example output:

```
0!! = 1
1!! = 1
2!! = 2
3!! = 3
4!! = 8
5!! = 15
6!! = 48
7!! = 105
8!! = 384
9!! = 945
```

Functions Month Calendar (*This is a grade level 5 exercise*)

Create a program called **functions_month_calendar**. The program should have a function called **print_month_calendar**. The function should take *year* and *month number* as arguments and print a *one-month calendar*. See the example output below for more details.

The *main function* should first input *year* and *month number* from the user and then call the **print_month_calendar** function.

Example output:

```
Enter year: 2024
Enter month: 12

 > December 2024 <
 Mon Tue Wed Thu Fri Sat Sun
                           1
   2   3   4   5   6   7   8
   9  10  11  12  13  14  15
  16  17  18  19  20  21  22
  23  24  25  26  27  28  29
  30  31
```

INSTRUCTION (*please read the below carefully*)

You can use the *lookup list* below to display the month name based on the month number.

```
month_names = ["", "January", "February", "March", "April", "May","June",
    "July", "August", "September", "October", "November", "December"]
```

You can use the **date**() function of the **datename** module[1] to create a date object. In the example below, the date is *1 December 2021*.

```
from datetime import date

my_date = date(2021, 12, 1)
```

You can use the **monthrange**() function of the **calendar** module[2] determine the ***number of days in a month*** as below.

```
from calendar import monthrange

days_in_the_month = monthrange(my_date.year, my_date.month)[1]
```

You can determine the ***day of week*** as below (1 = Monday , 2 = Tuesday, ...).

```
day_of_week = my_date.weekday() + 1
```

In this exercise, the easiest way ***to print a day number*** is as below.

```
print(f"{day:4d}")
```

Hint 1 You can write a separate for loop to *print the first line of day numbers only*. Then you can write another loop to print the other lines of day numbers.

Hint 2 Please notice that there is *one leading space* on each calendar line. That is, ***all*** *day numbers can be printed with the same format specifier*.

[1] The **datetime** module is described on https://docs.python.org/3/library/datetime.html
[2] The **calendar** module is described on https://docs.python.org/3/library/calendar.html